

A Brain-Friendly Guide

Head First C



Discover the secrets
of the C coding gurus



Learn how make can
change your life



Avoid
embarrassing
pointer
mistakes

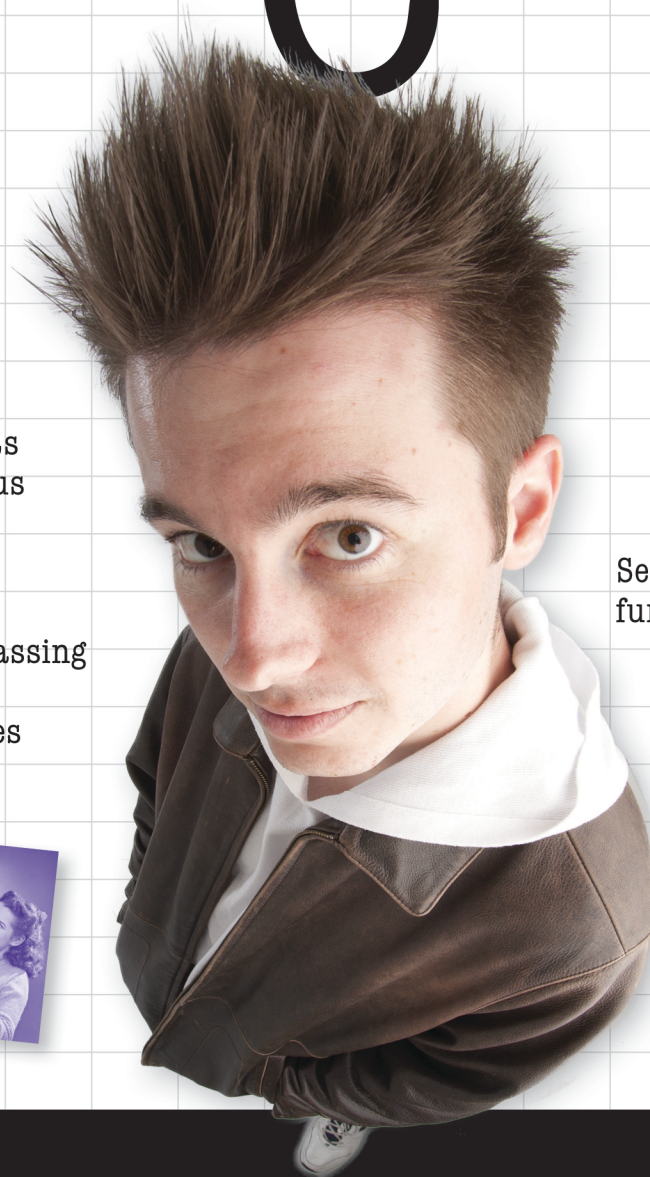
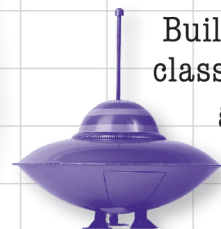
See how variadic
functions helped
Sue be more
flexible



Fool around
in the C
Standard
Library



Build a retro
classic arcade
game



O'REILLY®

David Griffiths &
Dawn Griffiths

1 getting started with c

Diving in



Want to get inside the computer's head?

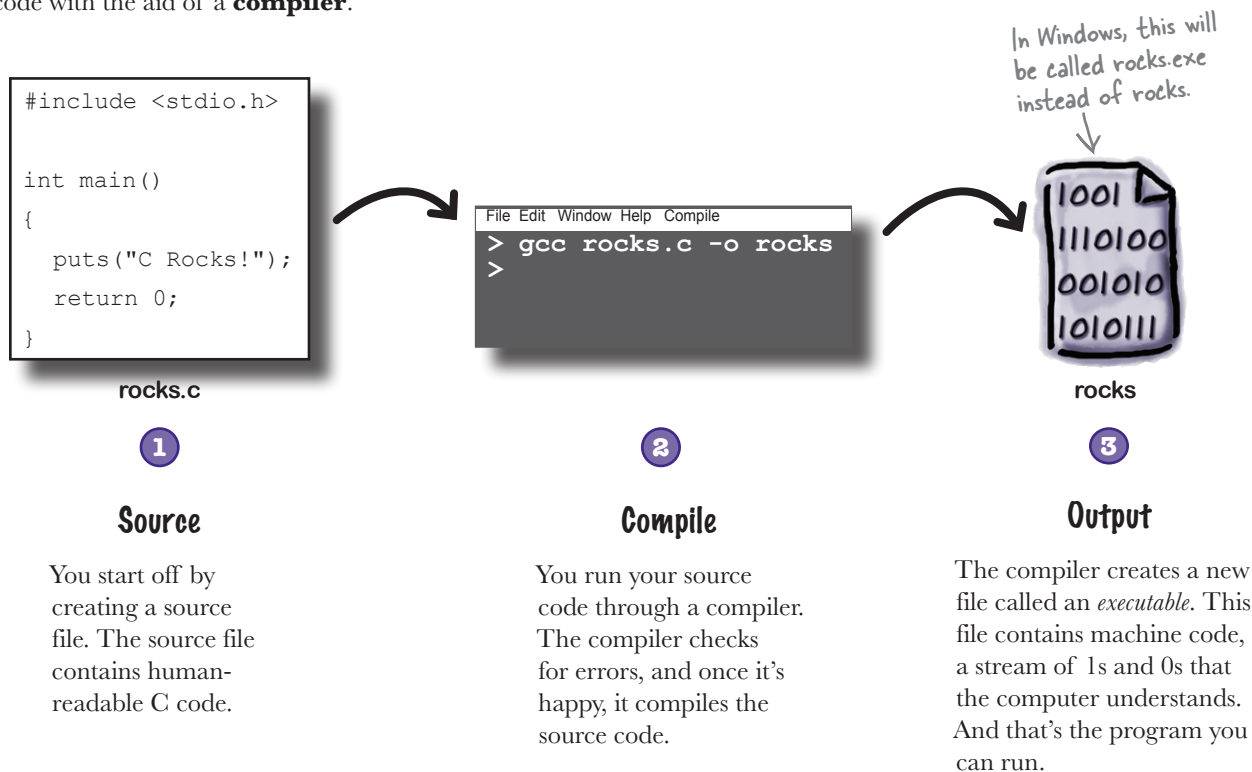
Need to write **high-performance code** for a new game? Program an **Arduino**? Or use that advanced **third-party library** in your iPhone app? If so, then C's here to help. C works at a **much lower level** than most other languages, so understanding C gives you a much better idea of **what's really going on**. C can even help you better understand other languages as well. So dive in and grab your compiler, and you'll soon get started in no time.

C is a language for small, fast programs

The C language is designed to create small, fast programs. It's lower-level than most other languages; that means *it creates code that's a lot closer to what machines really understand.*

The way C works

Computers really only understand one language: machine code, a binary stream of 1s and 0s. You convert your C code into machine code with the aid of a **compiler**.



C is used where speed, space, and portability are important. Most operating systems are written in C. Most other computer languages are also written in C. And most game software is written in C.

There are three C standards that you may stumble across. ANSI C is from the late 1980s and is used for the oldest code. A lot of things were fixed up in the C99 standard from 1999. And some cool new language features were added in the current standard, C11, released in 2011. The differences between the different versions aren't huge, and we'll point them out along the way.



Try to guess what each of these code fragments does.

Describe what you think the code does.



```
int card_count = 11;
if (card_count > 10)
    puts("The deck is hot. Increase bet.");
```

.....

.....

.....

```
int c = 10;
while (c > 0) {
    puts("I must not write code in class");
    c = c - 1;
}
```

.....

.....

.....

.....

```
/* Assume name shorter than 20 chars. */
char ex[20];
puts("Enter boyfriend's name: ");
scanf("%19s", ex);
printf("Dear %s.\n\n\tYou're history.\n", ex);
```

.....

.....

.....

.....

```
char suit = 'H';
switch(suit) {
case 'C':
    puts("Clubs");
    break;
case 'D':
    puts("Diamonds");
    break;
case 'H':
    puts("Hearts");
    break;
default:
    puts("Spades");
}
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

But what does a complete C program look like?

To create a full program, you need to enter your code into a *C source file*. C source files can be created by any text editor, and their filenames usually end with *.c*.

← This is just a convention, but you should follow it.

Let's have a look at a typical C source file.

1 C programs normally begin with a comment.

The comment describes the purpose of the code in the file, and might include some license or copyright information. There's no absolute need to include a comment here—or anywhere else in the file—but it's good practice and what most C programmers will expect to find.

The comment starts with `/*`.
These `*`s are optional. They're only there to make it look pretty.

The comment ends with `*/`.

```
/*
 * Program to calculate the number of cards in the shoe.
 * This code is released under the Vegas Public License.
 * (c)2014, The College Blackjack Team.
 */
```

2 Next comes the include section.

C is a very, very small language and it can do almost nothing without the use of *external libraries*. You will need to tell the compiler what external code to use by including header files for the relevant libraries. The header you will see more than any other is *stdio.h*. The `stdio` library contains code that allows you to read and write data from and to the terminal.

```
#include <stdio.h>
```

```
int main()
{
    int decks;
    puts("Enter a number of decks");
    scanf("%i", &decks);
    if (decks < 1) {
        puts("That is not a valid number of decks");
        return 1;
    }
    printf("There are %i cards\n", (decks * 52));
    return 0;
}
```

3 The last thing you find in a source file are the functions.

All C code runs inside functions. The most important function you will find in any C program is called the **main() function**. The `main()` function is the starting point for all of the code in your program.

So let's look at the `main()` function in a little more detail.



The main() Function Up Close

The computer will start running your program from the `main()` function. The name is important: if you don't have a function called `main()`, your program won't be able to start.

The `main()` function has a **return type** of `int`. So what does this mean? Well, when the computer runs your program, it will need to have some way of deciding if the program ran successfully or not. It does this by checking the *return value* of the `main()` function. If you tell your `main()` function to return 0, this means that the program was successful. If you tell it to return any other value, this means that there was a problem.

This is the return type. It should always be `int` for the `main()` function.

Because the function is called "main," the program will start here.

If we had any parameters, they'd be mentioned here.

The body of the function is always surrounded by braces.

```
int main()
{
    int decks;
    puts("Enter a number of decks");
    scanf("%i", &decks);
    if (decks < 1) {
        puts("That is not a valid number of decks");
        return 1;
    }
    printf("There are %i cards\n", (decks * 52));
    return 0;
}
```

The function name comes after the return type. That's followed by the function parameters if there are any. Finally, we have the *function body*. The function body **must** be surrounded by *braces*.



Geek Bits

The `printf()` function is used to display **formatted output**. It replaces format characters with the values of variables, like this:

The first parameter will be inserted here as a string.

First parameter

```
printf("%s says the count is %i", "Ben", 21);
```

The second parameter will be inserted here as an integer.

Second parameter

You can include as many parameters as you like when you call the `printf()` function, but make sure you have a matching % format character for each one.

If you want to check the exit status of a program, type:

```
echo %ErrorLevel%
```

in Windows, or:

```
echo $?
```

in Linux or on the Mac.



Code Magnets

The College Blackjack Team was working on some code on the dorm fridge, but someone mixed up the magnets! Can you reassemble the code from the magnets?

Enter two characters
for the card name.

```

/*
 * Program to evaluate face values.
 * Released under the Vegas Public License.
 * (c)2014 The College Blackjack Team.
 */

.....

.....

....._main()
{
    char card_name[3];
    puts("Enter the card_name: ");
    scanf("%2s", card_name);
    int val = 0;
    if (card_name[0] == 'K') {
        val = 10;
    } else if (card_name[0] == 'Q') {

        .....

    } else if (card_name[0] == ..... ) {
        val = 10;

    } ..... (card_name[0] == ..... ) {

        .....

    } else {
        val = atoi(card_name);
    }
    printf("The card value is: %i\n", val);
    .....0;
}

```

<stdlib.h> ;
; val = 11
int 'J'
#include 'A'

This converts the
text into a number.

return
else #include
if val = 10
<stdio.h>

But how do you run the program?

C is a *compiled language*. That means the computer will not interpret the code directly. Instead, you will need to convert—or *compile*—the human-readable source code into machine-readable *machine code*.

To compile the code, you need a program called a **compiler**. One of the most popular C compilers is the *GNU Compiler Collection* or **gcc**. gcc is available on a lot of operating systems, and it can compile lots of languages other than C. Best of all, it's completely free.

Here's how you can compile and run the program using gcc.

- 1 Save the code from the Code Magnets exercise on the opposite page in a file called `cards.c`.



cards.c

← C source files usually end `.c`.

- 2 Compile with `gcc cards.c -o cards` at a command prompt or terminal.

Compile `cards.c`
to a file called `cards`.

```
File Edit Window Help Compile
> gcc cards.c -o cards
>
```



cards.c



cards

↑
This will be `cards.exe`
if you're on Windows.

- 3 Run by typing `cards` on Windows, or `./cards` on Mac and Linux machines.

```
File Edit Window Help Compile
> ./cards
Enter the card_name:
```



Geek Bits

You can compile and run your code on most machines using this trick:

Here means "and then if it's successful, do this..."

`gcc zork.c -o zork && ./zork`

↓ You should put "zork" instead of "./zork" on a Windows machine.

This command will run the new program only if it compiles successfully. If there's a problem with the compile, it will skip running the program and simply display the errors on the screen.



You should create the `cards.c` file and compile it now. We'll be working on it more and more as the chapter progresses.



Test Drive

Let's see if the program compiles and runs. Open up a command prompt or terminal on your machine and try it out.

This line compiles the code and creates the cards program.

This line runs the program. If you're on Windows, don't type the ./

Running the program again

The user enters the name from a card...

...and the program displays the corresponding value.

```
File Edit Window Help 21
> gcc cards.c -o cards
> ./cards
Enter the card_name:
Q
The card value is: 10
> ./cards
Enter the card_name:
A
The card value is: 11
> ./cards
Enter the card_name:
7
The card value is: 7
```

Remember: you can combine the compile and run steps together (turn back a page to see how).

The program works!

Congratulations! You have compiled and run a C program. The `gcc` compiler took the human-readable source code from `cards.c` and converted it into computer-readable *machine code* in the `cards` program. If you are using a Mac or Linux machine, the compiler will have created the machine code in a file called **`cards`**. But on Windows, all programs need to have a `.exe` extension, so the file will be called **`cards.exe`**.

there are no Dumb Questions

Q: Why do I have to prefix the program with `./` when I run it on Linux and the Mac?

A: On Unix-style operating systems, programs are run only if you specify the directory where they live or if their directory is listed in the `PATH` environment variable.